

System Requirements Specification (SRS)

Project Title: AI-Powered Learning Assistant

Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed overview of the "AI-Powered Learning Assistant." This system uses Retrieval-Augmented Generation (RAG) to allow users to interact with uploaded PDF documents, ask questions, and take AI-generated quizzes to test their knowledge.

1.2 Scope

This system includes a web-based frontend, a backend server for orchestration, a vector database for semantic search, and an LLM integration for generating natural language responses and assessments.

2. System Architecture

The system follows a modern AI-stack architecture:

- **Frontend:** React.js / Next.js (User Interface)
- **Backend:** Node.js (Express) or Python (FastAPI)
- **Primary Database:** MongoDB / PostgreSQL (User profiles and scores)
- **Vector Database:** Pinecone or ChromaDB (Document embeddings)
- **AI Layer:** LLM (Gemini/GPT) via RAG pipeline

3. Functional Requirements

3.1 User Authentication

- **FR-1:** Users must be able to register and log in via email/password or OAuth.
- **FR-2:** The system shall maintain session persistence using JWT or cookies.

3.2 Document Management

- **FR-3:** Users shall be able to upload PDF files.
- **FR-4:** The system shall extract text from the PDF and split it into manageable chunks.
- **FR-5:** The system shall convert text chunks into embeddings and store them in the Vector Database.

3.3 RAG-based Q&A

- **FR-6:** The user shall be able to submit text queries related to the uploaded PDF.
- **FR-7:** The system shall retrieve the most relevant document chunks based on vector similarity.
- **FR-8:** The system shall provide a context-aware answer using the LLM.

3.4 Automated Quiz Generation

- **FR-9:** Users shall have the option to generate a quiz based on the document content.
- **FR-10:** The system shall generate Multiple Choice Questions (MCQs) in JSON format.
- **FR-11:** The system shall track user answers, calculate a score, and store it in the primary database.

4. Non-Functional Requirements

4.1 Performance

- **NFR-1:** Document processing (embedding) should complete within 10–15 seconds for a standard 10-page PDF.
- **NFR-2:** Chat responses should be streamed or returned within 3 seconds.

4.2 Security

- **NFR-3:** User PDFs must be private and accessible only by the owner.
- **NFR-4:** API keys for the LLM must be stored securely in server-side environment variables.

4.3 Scalability

- **NFR-5:** The vector database should handle multiple concurrent document indexes efficiently.

5. Data Requirements

5.1 Data Dictionary

Entity	Attributes
User	ID, Email, PasswordHash, CreatedAt
Document	ID, UserID, FileName, VectorNamespace
QuizScore	ID, UserID, DocumentID, Score, TotalQuestions

6. Proposed API Endpoints

- `POST /api/auth/register` - User registration.
- `POST /api/docs/upload` - PDF upload and embedding trigger.
- `POST /api/chat/ask` - Send query, get RAG-based answer.
- `GET /api/quiz/generate` - Retrieve AI-generated MCQs.
- `POST /api/quiz/submit` - Save score to the database.